

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

**Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»**

ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ И КОМПЬЮТЕРНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине

‘ВЫЧИСЛИТЕЛЬНАЯ МАТЕМАТИКА’

Вариант №5

Выполнил:
Студент группы Р3210
Давидюк А.Ю.

Преподаватель:
Рыбаков С.Д.

Санкт-Петербург, 2026

Цель работы

Изучить численные методы решения систем линейных алгебраических уравнений и реализовать один из них средствами программирования.

Описание метода

Метод Гаусса с выбором главного элемента по столбцам.

Схема с выбором главного элемента является одной из модификаций метода Гаусса. Идеей является такая перестановка уравнений, чтобы на k -ом шаге исключения ведущим элементом a_{ii} оказывался наибольший по модулю элемент k -го столбца.

Расчетные формулы

Прямой ход (приведение к треугольному виду):

1. **Выбор главного элемента** на k -м шаге: $|a_{i_k,k}| = \max_{i=k,\dots,n} |a_{i,k}|$
2. **Перестановка строк**: строка с номером k меняется местами со строкой i_k
3. **Исключение неизвестных**: $m_{ik} = \frac{a_{ik}}{a_{kk}}, i = k + 1, \dots, n, a_{ij}^{(new)} = a_{ij} - m_{ik} \cdot a_{kj}, j = k, \dots, n + 1$

Обратный ход (нахождение решения):

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij} x_j}{a_{ii}}, i = n - 1, \dots, 1$$

Вычисление определителя:

$$\det(A) = (-1)^p \cdot \prod_{i=1}^n a_{ii}$$

где p — количество перестановок строк.

Листинг программы

```
import os
import numpy as np
import copy

def read_matrix():
    choice = ""
    while choice not in ["f", "k", "r"]:
        choice = input("Введите матрицу: f - файл, k - клавиатура, r - случайная: ").strip().lower()

    if choice == "f":
        file_path = input("Путь к файлу: ")
        while not os.path.isfile(file_path):
            file_path = input("Файл не найден. Введите путь снова: ")
        with open(file_path, 'r', encoding='utf-8') as f:
            lines = [line.strip() for line in f if line.strip()]
            n = int(lines[0])
            matrix = [list(map(float, line.split())) for line in lines[1:n+1]]

    elif choice == "k":
        n = int(input("Введите количество неизвестных(от 1 до 20): "))
        print("Введите коэффициенты при неизвестных и свободные члены в конце каждой строки")
        matrix = []
        for i in range(n):
            row = list(map(float, input().split()))
            while len(row) != n + 1:
                print(f"Ошибка: требуется {n+1} чисел. Повторите ввод строки {i+1}:")
            row = list(map(float, input().split()))
            matrix.append(row)

    else:
        n = int(input("Введите количество неизвестных(от 1 до 20): "))
        import random
        matrix = [[random.uniform(-20, 20) for _ in range(n + 1)] for _ in range(n)]

    return matrix, n

def gauss_method(matrix, n):
    original_matrix = copy.deepcopy(matrix)
    swaps = 0

    for col in range(n - 1):
        max_row = col
        for row in range(col + 1, n):
            if abs(matrix[row][col]) > abs(matrix[max_row][col]):
                max_row = row
```

```

    if max_row != col:
        matrix[col], matrix[max_row] = matrix[max_row], matrix[col]
        swaps += 1

    pivot = matrix[col][col]
    if abs(pivot) < 1e-12:
        print("Матрица вырождена. Решение невозможно.")
        return None, None, None, 0

    for row in range(col + 1, n):
        factor = matrix[row][col] / pivot
        for k in range(col, n + 1):
            matrix[row][k] -= factor * matrix[col][k]

    det = (-1) ** swaps
    for i in range(n):
        det *= matrix[i][i]

    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        sum_known = sum(matrix[i][j] * x[j] for j in range(i + 1, n))
        x[i] = (matrix[i][n] - sum_known) / matrix[i][i]

    residuals = []
    for i in range(n):
        val = sum(original_matrix[i][j] * x[j] for j in range(n)) -
original_matrix[i][n]
        residuals.append(val)

    return matrix, x, residuals, det

def compare_with_numpy(matrix, n):
    A = [row[:n] for row in matrix]
    b = [row[n] for row in matrix]
    print("\nСравнение с библиотекой numpy:")
    try:
        x_np = np.linalg.solve(A, b)
        det_np = np.linalg.det(A)
        print("Решение системы:", *[f"{v:.10f}" for v in x_np])
        print(f"Определитель: det = {det_np:.5f}")
    except np.linalg.LinAlgError:
        print("det = 0")

def main():
    matrix, n = read_matrix()

    print("\nИсходная матрица:")
    for row in matrix:
        print(" ".join(f"{v:10.5f}" for v in row))

```

```

tri_matrix, solution, residuals, det = gauss_method(copy.deepcopy(matrix), n)

if tri_matrix is None:
    return

print("\nРешение методом Гаусса:")
for i, val in enumerate(solution):
    print(f"x[{i+1}]= {val:.25f}")

print("\nПреобразованная матрица:")
for row in tri_matrix:
    print(" ".join(f"{v:10.5f}" for v in row))

print(f"\nОпределитель: {det:.5f}")

print("\nВектор невязки:")
for i, val in enumerate(residuals):
    print(f"r[{i+1}]= {val:.25f}")

compare_with_numpy(tri_matrix, n)

if __name__ == "__main__":
    main()

```

Примеры работы программы

```
Введите матрицу: f - файл, k - клавиатура, r - случайная: f
Путь к файлу: test1.txt

Исходная матрица:
1000000000000.00000    1.00000    2.00000    3.00000    4.00000    5.00000
   6.00000    7.00000    8.00000    9.00000    1.00000    2.00000
   2.00000    3.00000    5.00000    76.00000    87.00000    8.00000
   2.00000    3.00000    5.00000    7.00000    8.00000    9.00000
   1.00000    2.00000    4.00000    7.00000    89.00000    9.00000

Решение методом Гаусса:
x[1]=-0.0000000000002712140341552
x[2]=-5.6742640707027156565800396
x[3]=5.2248322147651977687132785
x[4]=-0.0081400757525420357196833
x[5]=-0.0055485414313240447731213

Преобразованная матрица:
1000000000000.00000    1.00000    2.00000    3.00000    4.00000    5.00000
   0.00000    7.00000    8.00000    9.00000    1.00000    2.00000
   0.00000    0.00000    1.71429    4.42857    88.71429    8.42857
   0.00000    0.00000    0.00000    68.08333    5.25000    -0.58333
   0.00000    0.00000    0.00000    0.00000    -73.67931    0.40881

Определитель: -6019599999967168.00000

Вектор невязки:
r[1]=0.0000000000000000000000000000000
r[2]=0.00000000000000000000008881784197
r[3]=-0.00000000000000017763568394
r[4]=-0.00000000000000017763568394
r[5]=-0.00000000000000017763568394

Сравнение с библиотекой numpy:
Решение системы: 0.0000000000 -5.6742640707 5.2248322148 -0.0081400758 -0.0055485414
Определитель: det = -6019599999967360.00000
```

```

Введите матрицу: f - файл, k - клавиатура, r - случайная: r
Введите количество неизвестных(от 1 до 20): 4

Исходная матрица:
10.26589 -7.89502 -6.22391 4.64166 -16.62476
-7.28009 -5.53155 1.30177 -16.48193 15.36005
10.27833 9.67938 16.85356 -14.45335 19.67800
-12.13349 8.17492 -0.34335 -13.94454 16.31150

Решение методом Гаусса:
x[1]=-0.6745022997180353252488771
x[2]=0.0627722844885157948979781
x[3]=1.0523914158354290560026811
x[4]=-0.5719521505349814960439403

Преобразованная матрица:
-12.13349 8.17492 -0.34335 -13.94454 16.31150
0.00000 16.60438 16.56270 -26.26583 33.49554
0.00000 0.00000 11.91808 -24.62430 26.62641
0.00000 0.00000 0.00000 -20.14742 11.52336

Определитель: 48376.50033

Вектор невязки:
r[1]=0.0000000000000035527136788
r[2]=-0.0000000000000035527136788
r[3]=0.000000000000000000000000
r[4]=0.000000000000000000000000

Сравнение с библиотекой numpy:
Решение системы: -0.6745022997 0.0627722845 1.0523914158 -0.5719521505
Определитель: det = 48376.50033

```

Вывод

В результате выполнения данной лабораторной работой я познакомился с различными численными методами решения математических задач на примере систем алгебраических уравнений, реализовав на языке программирования Python метод Гаусса с выбором главного элемента по столбцам. Результаты, полученные моей программой, почти не отличаются от результатов, полученных при вычислении с помощью библиотеки numpy. В сравнении с итерационными методами, в данном методе при увеличении порядка системы количество действий стремительно растёт, однако не накапливается погрешность.